EduRAG

Serverless AI Knowledge Base

Milestone 2 — Final Presentation

Anna Ostrowska • Michał Kukla • Norbert Frydrysiak

Serverless & Cloud Computing

June 2026

Agenda

- Project Overview
- Architecture
- Tech Stack
- Storage
- API Design
- SLA / SLO / SLI
- Large-Scale Assumptions
- Key Features
- Delivery Assessment
- Live Demo
- Summary

Project Overview

What is EduRAG?

EduRAG is a cloud-native, serverless knowledge base that lets students **chat with their study materials** using AI.

Core RAG loop

Upload a PDF $\Rightarrow$ automatic indexing $\Rightarrow$ ask in natural language $\Rightarrow$ answer with **page citations**

Built on **Google Cloud Platform** — fully event-driven, PaaS/Serverless.

Key Parameters

LLM	Gemini 2.5 Flash
Embeddings	text-embedding-004
Vector dims	768
Chunk size	1,000 chars
Chunk overlap	100 chars
Top-K	3 chunks
Max PDF	50 MB
History	last 3 turns

User Roles

Student / End User

- Registers with username + password
- Uploads PDFs into **subject folders**
- Searches & filters documents
- Selects documents to query (checkboxes)
- Chats with streaming AI answers + citations
- Manages **persistent chat sessions**
- Deletes documents on demand

Administrator (implicit)

- No separate admin UI
- Infrastructure via **Terraform IaC**
- Access via GCP Console
- Firestore rules enforce isolation

Auth Model

Firebase Auth — ID token

`Authorization: Bearer {token}`

verified server-side on every request.

Architecture

System Architecture

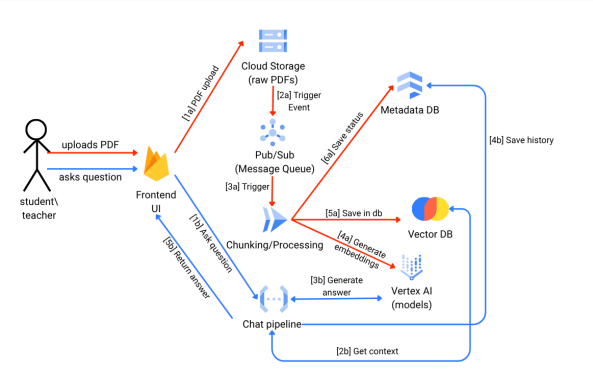

Figure 1: Async ingestion (red) & real-time inference (blue)

Pipeline 1 — Asynchronous Ingestion

1. Frontend requests a **pre-signed GCS URL** from the API
2. PDF uploaded **directly to GCS** — `uploads/{uid}/{subject}/{file}`
3. GCS finalize event → **Pub/Sub** message
4. **Worker (Cloud Run)** processes the message:
   - Downloads PDF; extracts text with `PyPDFLoader`
   - Splits into 1,000-char chunks (overlap 100)
   - Embeds chunks via **Vertex AI** `text-embedding-004`
   - Stores vectors + metadata in **ChromaDB**
   - Firestore: Uploading → Downloading → Chunking → Vectorizing → **Ready**
5. UI updates status in real-time via **Firestore listener**

Pipeline 2 — Real-Time Inference

1. User selects documents and submits a question
2. `POST /ask`: question, doc_ids[], history[]
3. API verifies Firebase ID token; embeds question via **Vertex AI**
4. **Smart retrieval:** question names a file ⇒ only that doc searched; otherwise all selected docs ⇒ top-3 chunks by cosine similarity
5. Prompt: System (context) + last 3 turns + question
6. **Gemini 2.5 Flash** streams tokens via **SSE**
   `data:{"token":"..."}` ··· `data:{"sources":[...],"done":true}`
7. UI renders **markdown** live; shows **citations**; session saved to Firestore

Tech Stack

Tech Stack

GCP Services

Compute	Cloud Run (API + Worker)
Messaging	Cloud Pub/Sub
Storage	Cloud Storage (GCS)
Database	Firestore (NoSQL)
Vectors	ChromaDB on Compute Engine
AI	Vertex AI (Gemini + Embeddings)
Auth	Firebase Auth
Hosting	Firebase Hosting
IaC	Terraform

Libraries & Frameworks

API	Flask + gunicorn
AI chain	LangChain
Embeddings	VertexAIEmbeddings
PDF	PyPDFLoader
Auth SDK	firebase-admin (Python)
Frontend	Vanilla JS + Tailwind CSS
Markdown	marked.js
Containers	Docker

IaaS exception

ChromaDB on Compute Engine VM — Vertex AI Vector Search too costly.

Storage

Storage Design

Store	Consistency	Role	Pattern
Cloud Storage	Strong	Raw PDFs. Path: `uploads/uid/subject/file`	Write-once; read-once by Worker
Firestore	Strong	Doc metadata + status; persistent chat sessions	High R/W; real-time listeners
ChromaDB	Eventual	Text embeddings + metadata (doc, user, subject, page)	Write-once; read-many on every query

Volume Estimate

10k users $\times$ 50 docs $\times$ 500 chunks $\times$ 768 floats $\approx$ **28 GB** vectors

GCS PDFs: up to **25 TB**

Document Deletion

Atomic across all 3 stores:

ChromaDB $\rightarrow$ GCS $\rightarrow$ Firestore

API Design

API Endpoints

Method	Endpoint	Description
`GET`	`/generate-upload-url`	Pre-signed GCS URL. Params: filename, subject. Pre-creates Firestore doc (status: Uploading).
`POST`	`/ask`	SSE streaming chat. Body: question, doc_ids[], history[]. Streams tokens, then sources.
`DELETE`	`/documents/{doc_id}`	Atomically removes vectors (ChromaDB), file (GCS), record (Firestore).

Auth — all endpoints

`Authorization: Bearer {token}`

Verified via `firebase-admin` SDK. Returns `401` if missing.

SSE stream format

```
data: {"token": "Hello"}
data: {"token": " world"}
data: {"sources":[...], "done":true}
```

SLA / SLO / SLI

SLA / SLO / SLI

SLA — commitment

99.9% uptime for Chat UI and Inference API, backed by GCP managed-service SLAs.

SLOs — objectives

- **Latency:** 95% of `/ask` requests — first token < 2 s
- **Ingestion:** 99% of PDFs ($<$50 MB) vectorised < 3 min
- **Availability:** error rate $< 0.1\%$ (30-day window)

SLIs — how we measure

- **Time-to-first-token:** Cloud Run latency to first SSE chunk
- **Ingestion duration:** Firestore delta Uploading $\rightarrow$ Ready
- **Error rate:** HTTP 5xx / total (Cloud Run metrics)

ChromaDB caveat

Eventual consistency — new chunks may not be queryable for a few seconds after ingestion.

Large-Scale Assumptions

Large-Scale Assumptions

Load model	
Registered users	10,000
Concurrent users	500
Docs per user	up to 50
Doc size	up to 50 MB
Chat requests	1,000 QPS
Uploads/hour	200

Why it scales

- **Cloud Run** — scale-to-zero, auto-scales under load
- **Pub/Sub** — buffers upload bursts; Worker consumes async
- **Direct-to-GCS** — binary transfer bypasses API entirely
- **Firestore** — thousands of concurrent real-time listeners

Bottleneck

ChromaDB on a **single VM** — sharding requires a managed vector DB.

Key Features

Key Features — Infrastructure & Security

Infrastructure

- Full **Terraform IaC** — Cloud Run, GCS, Pub/Sub, Firestore, IAM, Artifact Registry
- Event-driven: GCS → Pub/Sub → Worker
- Dockerised (Flask + gunicorn)
- Worker: 2 vCPU / 2 GiB RAM
- Dedicated Service Account, least-privilege IAM

Security

- **Firebase Auth** — username/password (`name@edurag.app` synthetic email)
- Per-user isolation: Firestore sub-collection + ChromaDB `user_id` filter
- **Pre-signed GCS URLs** — PDFs bypass backend
- Firebase ID token verified on every request
- CORS for Firebase Hosting origin

Key Features — Document Management

Upload & Organisation

- **Subject folders** — organise PDFs by topic
- Drag-and-drop or browse upload
- **Real-time status** (Firestore listener): Downloading → Chunking → Vectorizing → **Ready**
- New *Ready* docs auto-selected for chat
- **Atomic delete**: ChromaDB → GCS → Firestore

Search, Filter & Select

- **Search** by filename or subject
- **Subject dropdown** filter
- **Ready-only** toggle
- Per-document checkboxes to scope queries
- Subject-level **bulk toggle** (all/none)
- Global select-all / deselect-all
- Active selection shown as **chips**

Key Features — Chat Interface & Sessions

AI Chat

- **Streaming responses** (SSE) — token-by-token
- **Markdown** — bold, lists, code blocks
- **Source citations** — filename, page, excerpt
- **Smart routing** — mention a filename ⇒ only that document is searched
- **Multi-turn context** — last 3 turns
- Copy to clipboard; clear chat

Persistent Chat Sessions

- Multiple sessions per user (ChatGPT-style sidebar)
- Persisted in Firestore `users/{uid}/chats`
- Auto-title from first message; manual rename
- Delete session with confirmation
- Sorted by last activity
- Up to 50 messages stored per session

Delivery Assessment

Plan vs. Delivered — all km0 requirements met

km0 Requirements

Requirement	Met
Async ingestion pipeline	✓
Interactive Chat UI	✓
Scalable vector storage (3 tiers)	✓
Real-time processing status	✓
Terraform IaC	✓
Direct-to-cloud upload	✓
Event-driven decoupling (Pub/Sub)	✓
PaaS / Serverless preference	✓*

*ChromaDB on Compute Engine — Vertex AI Vector Search too costly

Beyond the original plan

- Persistent **multi-session chat** (Firestore)
- Chat session rename / delete
- Document **search & filter**
- Subject-level bulk selection
- **Smart filename routing**
- Expandable **source citations**
- Selected-doc **chips** bar

Originally: separate Admin/End-User roles. Delivered: unified role — each user manages their own documents and sessions.

Schedule & SLA / SLO — Status

Project Schedule

Date	Phase	
Apr 17	M0 Submission	✓
Apr 27	M1 Presentation	✓
May 01–22	Build	✓
May 23–Jun 02	Testing	✓
Jun 03	M2 Presentation	✓

SLA / SLO Assessment

Objective	Target	Met
Uptime	99.9%	✓
First-token latency (p95)	$<$ 2 s	✓
Ingestion ($<$50 MB PDF)	$<$ 3 min	✓
Error rate (30-day)	$<$ 0.1%	✓

SLIs: Cloud Run metrics + Firestore timestamps.
km1 target was *full response $<$4s*;
delivered: **first token $<$2s** via SSE.

Live Demo

Live Demo

edurag-495620.web.app

Upload	Chat	Sessions
No page reload. Firestore push — status updates live.	Token-by-token streaming from Gemini. Expandable citations.	Multiple named sessions. Persists across browser sessions.

Summary

Summary

Architecture

- Async ingestion: GCS → Pub/Sub → Cloud Run
- Inference: Vertex AI Gemini 2.5 Flash via SSE
- Markdown, source citations, multi-user isolation
- Full **Terraform IaC** — all infra as code

User Experience

- Subject folders, search, filter, bulk select
- Persistent multi-session chat (Firestore)
- Smart filename routing in queries
- Real-time status + atomic document deletion

By the numbers

GCP services	9 + Firebase
API endpoints	3
Storage tiers	3 (GCS / Chroma / Firestore)
LLM	Gemini 2.5 Flash
Vector dims	768
Chunk size	1,000 chars
Max PDF	50 MB

One IaaS component

ChromaDB on Compute Engine VM

Vertex AI Vector Search too costly.

Q & A

Thank you!

Anna Ostrowska • Michał Kukla • Norbert Frydrysiak

edurag-495620.web.app